19 費氏堆積

Part V Advanced Data Structures

本章定位

Fibonacci heap用延後整理讓meld與decrease-key有低攤銷成本。必須同時理解root/child環、失子mark、級聯cut與degree碰撞，再以同一勢能支付操作並用Fibonacci下界限制整理陣列大小。

前置知識

可合併堆積、攤銷分析與切割標記。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以根串列與 child list 表示較鬆散的 heap。
- 攤銷 merge、insert、decrease-key。
- 用 degree bound 證明 consolidate 成本。
- 19.1 Structure of Fibonacci heaps
- 19.2 Mergeable-heap operations
- 19.3 Decreasing a key and deleting a node
- 19.4 Bounding the maximum degree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

19.1 Structure of Fibonacci heaps

Fibonacci heap 是多棵 min-heap 有序樹的集合。每個非根x滿足parent.key≤x.key；root list保存所有根，H.min指向最小鍵根，H.n是總節點數。decrease-key/erase以有效handle指定節點，不包含先搜尋任意鍵的成本。

每節點保存key、degree、parent、child、left、right、mark。degree是當前child數，child指向一個child；同層以環狀雙向串列相連。本文把所有根mark清為false；非根mark=true表示自上次連到現父後已失去一個child。

普通鍵可重複，但handle identity不同。刪除後handle失效；meld消耗兩個輸入heap的所有權，輸入不能再各自修改同一批節點。這些條件讓指標與成本主張有明確對象。

環狀串列的常數次指標操作

移除x時先保存next=x.right。若x是唯一節點，就把該串列head設NIL；否則設x.left.right=x.right、x.right.left=x.left，必要時head=next。最後令x.left=x.right=x，並更新所在容器的head/degree。

兩個非空環A、B可直接串接。取a∈A、b∈B，先保存an=a.right、bp=b.left，再設

```
a.right=b; b.left=a
bp.right=an; an.left=bp
```

這只改兩條相鄰邊，沒有走訪全部節點；空環直接回傳另一環。把x加入root list前，先從原child ring移除、degree減1，再設x.parent=NIL、mark=false。若先覆寫x.right才嘗試走原環，可能遺失後繼。

19.2 Mergeable-heap operations

MAKE-HEAP建立空root list、min=NIL、n=0。INSERT建立degree0、parent/child=NIL、mark=false的單節點環，接到root list，比較更新min，n加1。MINIMUM只回傳min，空heap回傳NIL。

MELD把兩個root rings接起，min取兩個min較小者，n相加；不合併樹、不掃描節點。各環已有最小根，min比較即足夠。這些操作都是O(1)真實成本。

用足夠大的固定常數C取勢能$\Psi=C(t+2m)$，t為根數、m為標記節點數。對多個heap取勢能總和，初始空heap為0且始終非負。INSERT使t增加1、Ψ增加C，故O(1)攤銷；MELD保留根/標記總數，$\Delta\Psi=0$，故仍O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extract-Min 的完整次序

```
extract-min(H):
    z=H.min
    if z==NIL: return NIL
    while z.child ring is nonempty:
        detach one child x from z
        x.parent=NIL; x.mark=false
        append x to root list
    remove z from root list; H.n--
    if H.n==0: H.min=NIL
    else: consolidate roots by degree; recompute H.min
    invalidate z's heap handle; return z
```

移除最小根後，其child都成為根，因樹內heap order仍成立；其餘根本來不小於z。新min要在整理後的根中重新找，不能把舊z指標留著，也不能把整棵z子樹一併刪除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Consolidate 的碰撞迴圈

```
A[0..D] = NIL
while old root list is nonempty:
    x=detach one root; d=x.degree
    while A[d]!=NIL:
        y=A[d]; A[d]=NIL
        if y.key<x.key: swap(x,y)
        make y a child of x; y.parent=x; y.mark=false
        x.degree++; d=x.degree
    A[d]=x
rebuild root list from nonempty A slots
H.min = smallest-key root
```

每次link合併相同degree的根，把較大鍵根接到較小鍵下，保持heap order；d加1後必須再次查A。結束後每種degree至多一根。D須至少⌊log_φ n⌋，可按需擴充；錯用⌊log_2 n⌋未必放得下合法最大degree。

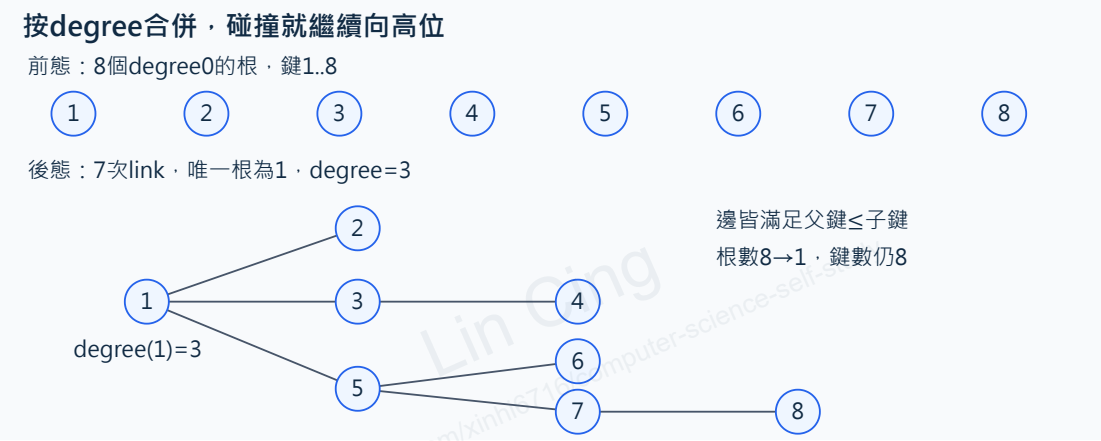
直接逐一消耗舊root ring時，degree陣列為O(log n)輔助空間。若用snapshot避免走訪時改鏈，則snapshot可能有Θ(n)根，不能還宣稱全部輔助空間只有O(log n)。

八個根的數值合併軌跡

依序插入0..8後extract0，留下8棵單節點根1..8，按此順序整理：

讀入根	這一輪link	完成後A中的根，以degree:key表示
1	無	0:1
2	2接到1	1:1
3	無	0:3，1:1
4	4接3；3接1	2:1
5	無	0:5，2:1
6	6接5	1:5，2:1
7	無	0:7，1:5，2:1
8	8接7；7接5；5接1	3:1

八個根的數值合併軌跡 (續)



7次link把根數8降到1，鍵集合仍是1..8。樹不必是完全樹；正確性依據是每條parent-child邊的heap order及實際degree。

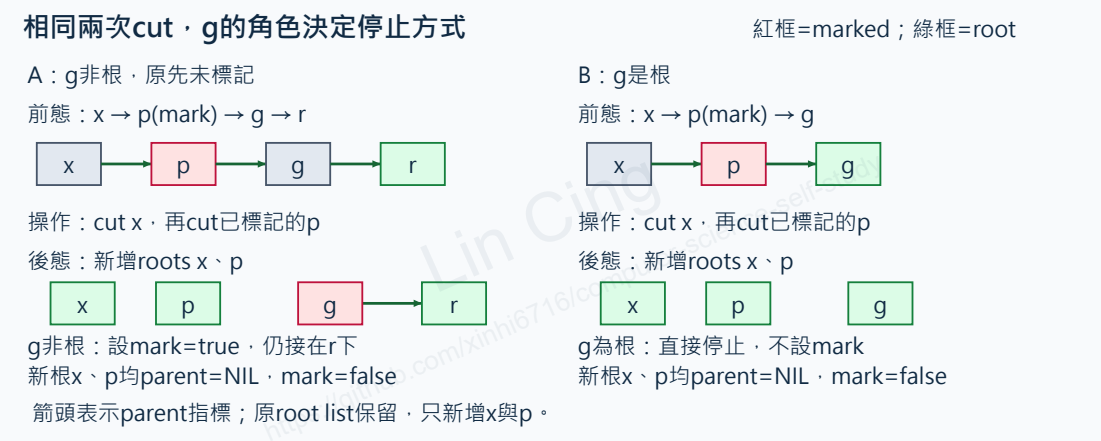
19.3 Decreasing a key and deleting a node

```
decrease-key(H,x,new):
    require new<=x.key and x is a valid handle
    x.key=new; p=x.parent
    if p!=NIL and x.key<p.key:
        cut x from p to root list
        cascading-cut(p)
    if x.key<H.min.key: H.min=x
cascading-cut(y):
    p=y.parent
    if p==NIL: return
    if y.mark==false: y.mark=true; return
    cut y from p to root list
    cascading-cut(p)
```

cut清除被切節點的mark，減少舊父degree並保持雙向環。降低x只可能破壞x與父的邊，與child的heap order反而更寬鬆；切斷違規邊後所有樹仍有序。mark規則另用來保護degree下界，不是為了比較最小鍵。

級聯切割的兩個停止分支

若p先前已標記，切走其第二個child x後，p也被切到root list。接著檢查p的舊父g：g非根且尚未標記，就設g.mark=true停止；g是根就直接停止，不因本次失子標記。



圖中root list是「新增x、p」，原本的根仍保留。`x.parent=NIL`、`p.parent=NIL`且mark=false；g若非根仍接在原父下，只改mark。若g也已標記，還要繼續cut g，不可套未標記分支。

合法操作歷史中的真實節點

從空heap插入0..64、extract0，得到64節點的已整理heap；節點id原本等於初始鍵，降鍵後id不改。

分支	第一次操作	第二次操作	第二次完成後
g是根	id6降−10，cut6，標記p=id5	id7降−11，cut7再cut5	roots={1,6,7,5}，g=id1不標記
g非根	id14降−10，cut14，標記p=id13	id15降−11，cut15再cut13	roots={1,14,15,13}，g=id9標記，仍在id1下

兩例第一次操作後t=2、m=1，t+2m=4。第二次根分支變t=4、m=0，勢能尺度仍4；非根分支t=4、m=1，變6。這些父子由同一組合法link、cut歷史產生，不是假定任意形狀一定能出現。

Decrease-Key 的完整勢能計費

若沒有cut，實際成本與勢能變化都是O(1)。若共cut $c \geq 1$個節點，根數增加c。除最先因鍵值違規被切的x外，其餘$c-1$個級聯cut節點在被切前都已標記，並在cut後清除；最後至多新標記一個未標記非根。

因此$\Delta m \leq -(c-1)+1 = 2-c$，

$\Delta(t+2m) \leq c+2(2-c) = 4-c$。

若實際成本$\leq ac+b$，固定$C \geq a$，則

$\hat{c} \leq ac+b+C(4-c) \leq b+4C = O(1)$。

這明確放大同一勢能$\Psi = C(t+2m)$，使釋放的信用支付實際指標操作常數。單次decrease-key可沿很長祖先鏈；常數界是任意操作序列的攤銷界。

Delete 的成本與特殊最小值

CLRS的抽象DELETE(x)先把x降為嚴格小於所有普通鍵的特殊$-\infty$，再extract-min，所以$O(1)+O(\log n) = O(\log n)$攤銷。不能把delete與decrease-key一起寫成O(1)。

$-\infty$是抽象鍵域的額外元素，不表示C++某整數型別的最小值一定可用。若heap原已允許同樣的最小值，平手可能讓extract-min刪掉別的handle。

也可直接刪指定handle：若x非根，先cut x並對舊父做cascading-cut；接著從root list移除這個x、把其children提升成根、consolidate並重算min。此流程毋須改key，仍是O(1)攤銷cut階段加$O(\log n)$攤銷根移除，不是省掉consolidate的O(1)刪除。

19.4 Bounding the maximum degree

固定節點x，當前k個child依它們連到x的時間先後排列為y1..yk。y1的degree≥0。對i≥2，yi當時連入x時，較早且現在仍存在的i−1個child都已在，所以x當時degree≥i−1。

link只合併相同degree的根，故yi當時degree也≥i−1。yi只要仍掛在x下，自那時起就至多失去一個child；失去第二個時早已被cut。因此現在

`degree(yi)≥i−2`。

這使用「當前仍存child的加入順序」，不是假設x從未失去child，也不是說現在所有child的degree都相同。

子樹大小至少是 Fibonacci 數

定F0=0、F1=1、F_(j+2)=F_(j+1)+F_j。證明任意合法節點x滿足size(x)≥F_(degree(x)+2)，以子樹節點數N做強歸納。

N=1時degree0、F2=1成立。N>1時，每個child子樹都小於N，歸納假設可用於其實際degree，不必假設child degree小於父degree。由上一頁對yi的界與F單調性，

`size(x)≥1+1+Σ_(i=2)^k F_i=F_(k+2)`，k≥1。

最後等式由Σ_(i=0)^k F_i=F_(k+2)−1配上F0=0、F1=1得到；k=1時空和也給2=F3。

再令ϕ=(1+√5)/2，F2=1=ϕ^0、F3=2≥ϕ；由前兩階相加，F_(k+2)≥ϕ^(k−1)+ϕ^(k−2)=ϕ^k。故n≥size(x)≥ϕ^k，得到`k≤log_ϕ n`。這是degree界，沒有把樹高宣稱為O(log n)。

Extract-Min 的勢能總帳

設移除前有t根，最小根至多D個child，移除並提升後至多t−1+D根。每次link根數減1，所以整個整理的實際成本O(t+D)，不會因while碰撞巢狀就變O(tD)。

整理後degree各不相同，至多D+1根，標記數不增加；因此$\Delta(t+2m)\le D+1-t$。在與decrease-key共用的足夠大C下，

$\hat{c}\le a(t+D)+b+C(D+1-t)=O(D+1)$。

由degree證明D=O(log(n+1))，extract-min與delete都是O(log(n+1))攤銷。minimum、insert、meld、decrease-key為O(1)攤銷，其中前三者還是O(1)實際成本。這些結論從同一非負勢能望遠鏡求和得到；有限操作測試只檢查圖、數值及實作狀態。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 19。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study